

Laboration 5

Uppgift 1

Beskriv om du har använt något hjälpmedel för att lösa uppgiften och hur du har använt detta hjälpmedel.

För denna uppgift har jag använt mig av LLM specifikt ChatGPT.

Jag har använt ChatGPT på ett sådant sätt att jag specificerar tydligt vad den skall göra för att kunna ge mig rätt resultat. För Edge och Vertex användes ChatGPT för att förstå hur man skall tänka, men det var ganska logiskt utifrån själva arbetsuppgiften vad som krävdes och vad som skulle göras.

Det var när jag väl kom till Graph delen som jag hade problem att förstå hur man skulle lösa saker och ting. Logiskt nog lät jag ChatGPT generera koden enligt specificering samt projektets beskrivning, därefter läste jag på om helheten gällande koden för att bygga en förståelse. Sedan innan vet jag att HashMap används för nyckel-värde-par, men jag har inte använt HashMap mycket, så behövde även använda ChatGPT för att förstå mer i djupet hur den fungerar med Edges och Vertices. Nu när jag väl förstod hur HashMap används i denna uppgift, så började jag kolla mer i de olika metoderna och jag lät även ChatGPT förklara saker och ting som jag inte förstod när jag väl kollade på koden.

```
@Override
public List<Edge<T>> getEdges(T info) {
    return edges.getDefault(info, new ArrayList<>());
}
```

Just denna del förstod jag inte mycket av, då jag inte visste vad exakt getDefault gör, men vet nu att den hämtar värdet för nyckel infon och om nyckel inte finns, så returnerar den en tom ArrayList istället.

Kortfattat: Har använt LLM för att bygga en förståelse och fråga frågor om det jag inte förstod, men även för att generera kod i Graph samt inspiration för kod.

Hur säkerställer du att flera Vertex-objekt med samma identifierare inte sparas i grafen? Kan det få existera två Vertex-objekt med samma identifierare?

Vi säkerställer detta genom HashMap, där vi vet att identifieraren används helt enkelt som en nyckel:

```
private HashMap<T, Vertex<T>> vertices;
```

Samt att innan vi lägger in en ny vertex, så kontrolleras det ifall identifieraren redan finns i addVertex metoden:

```
if (vertices.containsKey(info)) {  
    return;  
}
```

HashMap kan inte innehålla två likadana nycklar och därvid förhindras flera Vertex objekt med samma identifierare från att sparas i grafen.

Ifall det fanns flera Vertices med samma identifierare skulle det bli otydligt vilken vertex som är i fokus när vi väl använder till exempel,

```
vertices.get(info);
```

Detta skulle självklart leda till fel edges, vilket skulle leda till problem vid traversering och sökning i grafen.

Graf-klassen ska bara hantera oriktade kanter, finns det någon poäng med att ha riktade Edge-objekt?

Det är fortfarande en poäng att använda riktade Edge-objekt även fast Graph skall enbart hantera oriktade kanter, eftersom en oriktad kant kan representeras som två riktade kanter, det vill säga en i varje riktning:

A -> B | B -> A

Genom det, så görs implementationen enklare, eftersom varje Vertex då kan hålla koll på sina utgående kanter. Detta gör även att det blir enklare/lättare att traversera grafen för att hitta sina grannar till en vertex.

I min implementation skapas därför två stycken Edge-objekt i min addEdge metod.

```
Vertex<T> from = vertices.get(infoA);  
Vertex<T> to = vertices.get(infoB);
```

```
Edge<T> edge1 = new Edge<>(from, to);  
Edge<T> edge2 = new Edge<>(to, from);
```

Grafen fungerar helt enkelt som en oriktad, men samtidigt som varje enskild kant fortfarande är riktade internt.

Uppgift 2

Beskriv Dijkstras algoritm

Vi har använt Dijkstras algoritm i en tidigare kurs (SBG382). Denna algoritm används för att hitta den billigaste och snabbaste vägen till en nod genom ett nätverk eller graf i detta fall.

Viktiga begrepp:

- Startnod – Noden där algoritmen börjar sin sökning.
- Noder (vertices) - Punkterna i nätverket eller grafen som algoritmen traverserar mellan.
- Kanter (edges) - Kopplingarna mellan noderna.
- Vikter/Distance - Varje kant har ett avstånd eller en kostnad och detta representerar helt enkelt till exempel, en längd eller restid till den specifika noden.
- Besökta noder – Algoritmen håller koll på vilka noder som har redan behandlats.
- Kortaste avstånd - För varje nod sparas det kortaste avståndet hittills från startnoden.
- Föregående nod - Används för att kunna återskapa den kortaste vägen när algoritmen väl är färdig/klar.

Algoritmen fungerar på det sätt att den börjar med att sätta startnodens avstånd till 0 medan alla andra noder får ett oändligt avstånd, därefter väljs den nod som har lägst avstånd och som inte redan har besökts. Algoritmen kollar igenom alla grannar till noden, så att den alltid har koll på kostnaden för att se ifall en kortare väg kan hittas genom den aktuella noden. Ifall algoritmen lyckas hitta en kortare väg, så uppdateras nodens avstånd och dess föregående nod. Denna process fortsätter tills det att alla noder har besökts och då har algoritmen lyckats hitta den kortaste vägen från startnoden till alla andra noder i grafen.

Beskriv en bra graf-implementation för att utföra Dijkstras

Med tanke på att min kod har en implementation med Graph, Vertex och Edge, så kan Dijkstras algoritm utgå ifrån den. Vi vet att Grafen har vertices och edges samt att i edges används även ett avstånd i varje edge.

I själva Edge har vi ett attribut vid namn distance, och denna kan ju helt enkelt användas som en vikt i Dijkstras algoritm. Därvid behöver Edge egentligen inte fler attribut ifall avståndet mellan två noder är det som används som kostnad.

Dock i Vertex skulle det nog krävas några fler attribut för att hjälpa till i algoritmen, alltså en distans från startnoden, en boolean för besökta noder samt en som kollar föregående för att kunna återskapa den kortaste vägen efter att algoritmen är helt klar.

I min Graph skulle man även kunna lägga till en specifik metod för att ta emot startnodens och målnodens identifierare för att sedan returnera en lista med noder, alltså noder som representerar den kortaste vägen.

Typen T kan vara till exempel, String eller Integer och användas som ett unikt ID som kanske platsnamn eller nodnummer. Detta kan behövas, eftersom grafen använder info som nyckel för att hitta rätt vertex.

När vi vill visualisera algoritmens resultat kan man använda color attributen i Vertex samt Edge. Besökta noder kan färgas i en specifik färg och den kortaste vägen kan markeras genom att färga rätt noder och kanter i en specifik färg.